EXPERIMENT 10

Create a new MongoDB database and perform CRUD operations on it

Course Outcome	CO4	**Teaching Hours**	2

Objective

To perform CRUD (Create, Read, Update, Delete) operations on a MongoDB collection using mongosh.

Theory

CRUD operations form the core of interacting with any database. In MongoDB: Create uses insertOne()/insertMany(); Read uses find()/findOne() with optional query filters and projections; Update uses updateOne()/updateMany() with update operators like $set, $inc, $unset; Delete uses deleteOne()/deleteMany(). MongoDB queries use a flexible JSON-like syntax supporting comparison operators ($gt, $lt, $in, etc.) and logical operators ($and, $or).

Software / Hardware Requirements

MongoDB installed and running (from Experiment 10), mongosh.

Step-by-Step Procedure

1. Open mongosh and switch to a new database named storeDB.

```
use storeDB
```

2. Create (insert) multiple product documents into a products collection using insertMany().

```
db.products.insertMany([
  { name: "Laptop", category: "Electronics", price: 55000, stock: 12 },
  { name: "Mouse", category: "Electronics", price: 500, stock: 100 },
  { name: "Desk Chair", category: "Furniture", price: 4500, stock: 20 },
  { name: "Notebook", category: "Stationery", price: 40, stock: 300 }
])
```

3. Read all documents using find() and pretty-print them.

```
db.products.find().pretty()
```

4. Read with a filter: find all products in the 'Electronics' category.

```
db.products.find({ category: "Electronics" })
```

5. Use comparison operators: find all products priced above 1000.

```
db.products.find({ price: { $gt: 1000 } })
```

6. Use findOne() to retrieve just a single matching document, and a projection to return only selected fields.

```
db.products.findOne({ name: "Laptop" }, { name: 1, price: 1, _id: 0 })
```

7. Update a single document: reduce the price of 'Mouse' by using the $set operator.

```
db.products.updateOne({ name: "Mouse" }, { $set: { price: 450 } })
```

8. Update multiple documents at once: increase stock by 10 for every Electronics item using $inc.

```
db.products.updateMany({ category: "Electronics" }, { $inc: { stock: 10 } })
```

9. Delete a single document: remove the 'Notebook' product.

```
db.products.deleteOne({ name: "Notebook" })
```

10. Delete multiple documents matching a condition: remove all products with stock less than 5 (first insert one such test document, then delete it).

```
db.products.insertOne({ name: "Old Item", category: "Misc", price: 10, stock: 2 })

db.products.deleteMany({ stock: { $lt: 5 } })
```

11. Count total remaining documents in the collection using countDocuments().

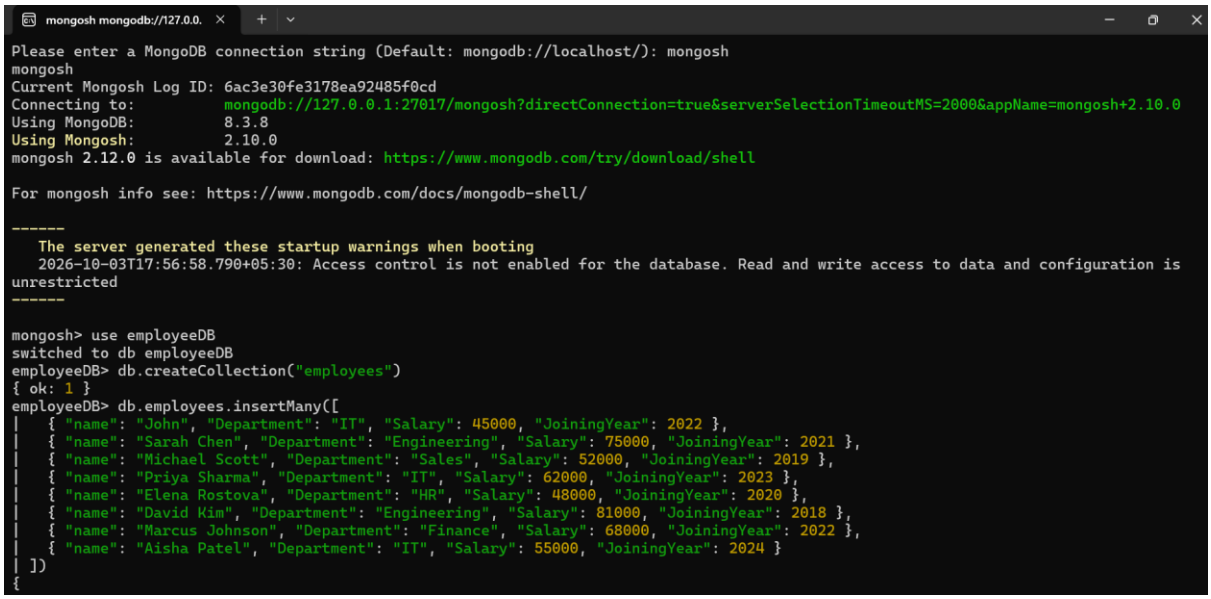
```
db.products.countDocuments()
```

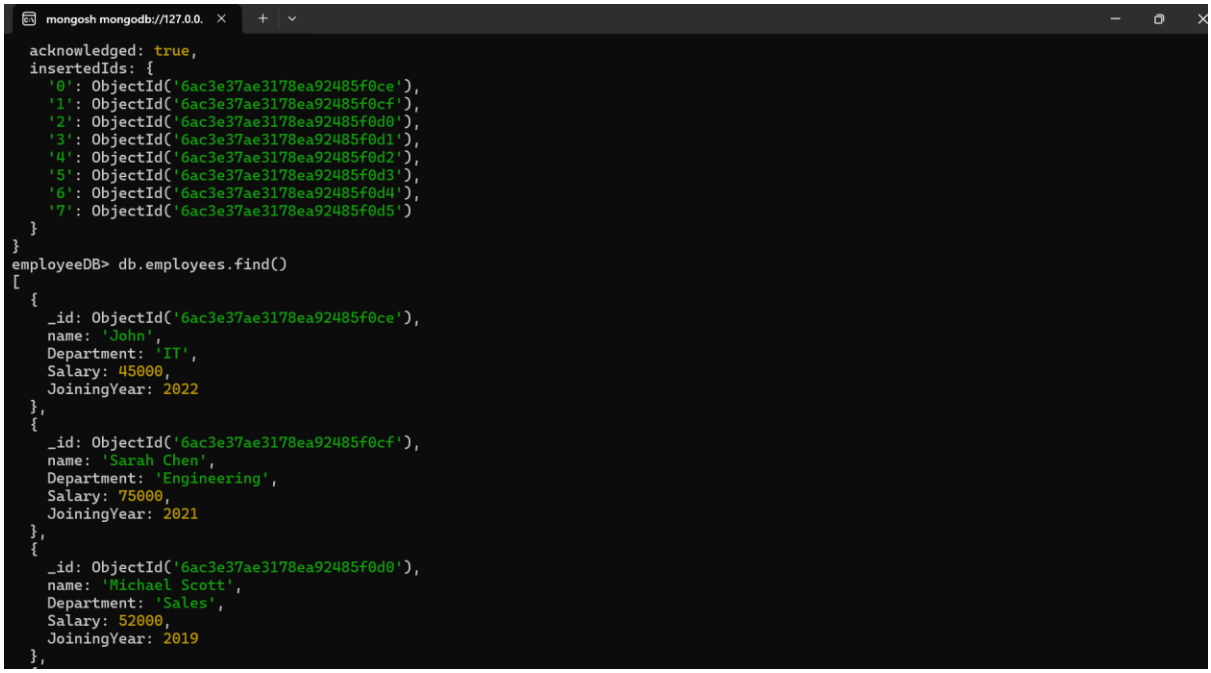
12. Sort results by price descending and limit to top 2 most expensive products.

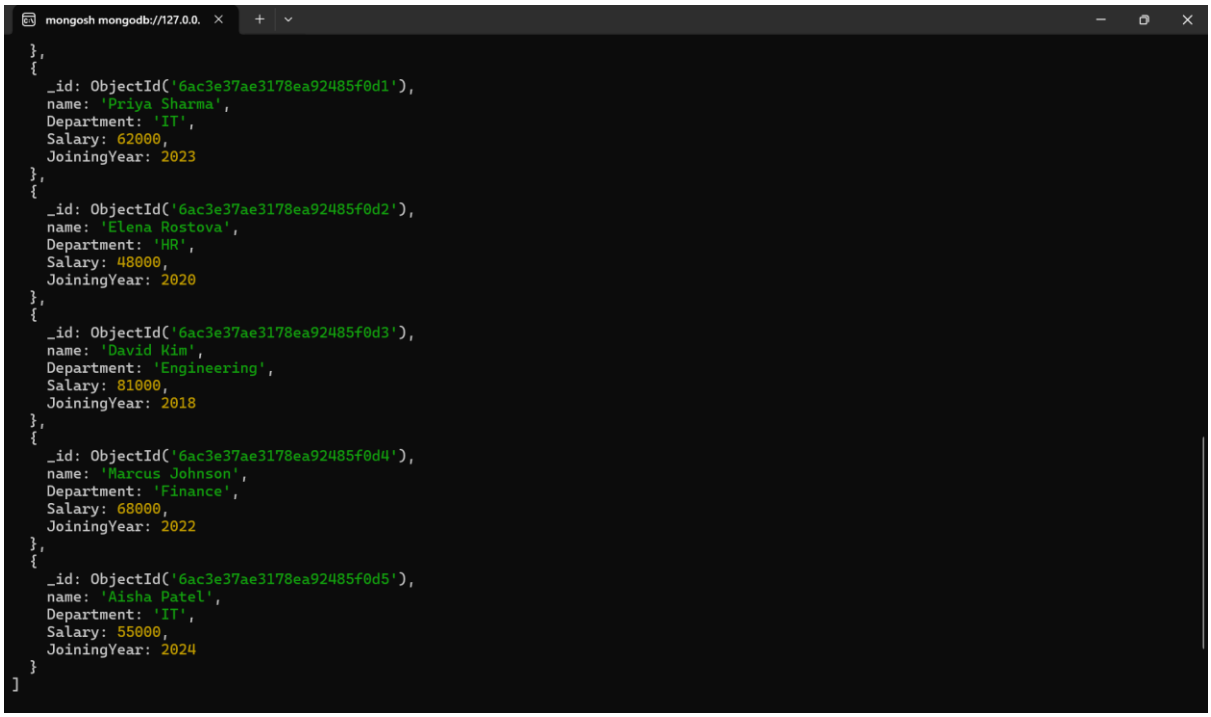
```
db.products.find().sort({ price: -1 }).limit(2)
```

Hands-on Assignment

1. Create a new database employeeDB with a collection employees containing at least 8 documents with fields: name, department, salary, and joiningYear.



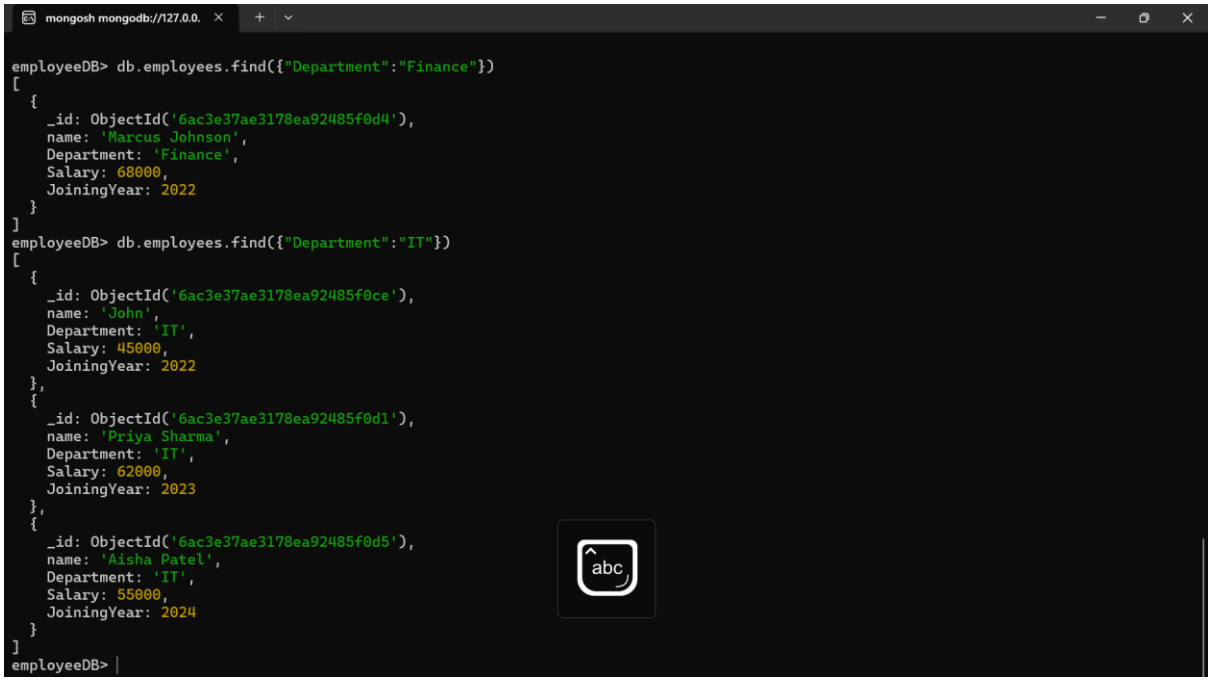




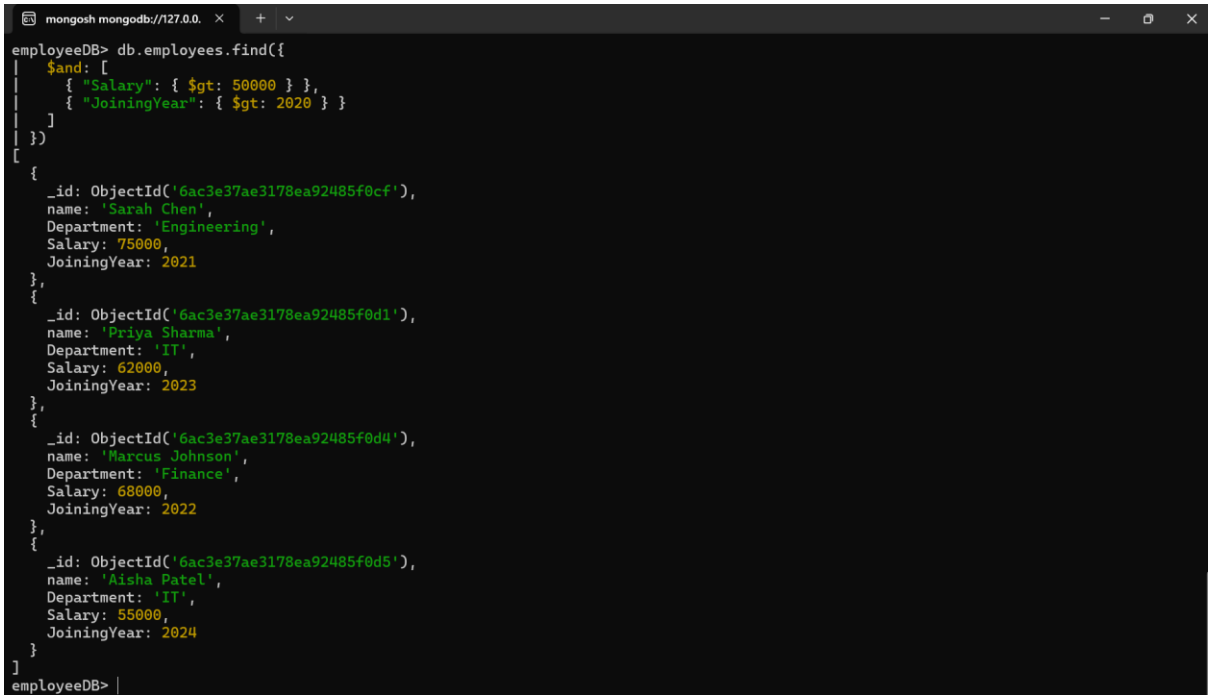
2. Write and execute queries to: (a) find all employees in a specific department, (b) find employees with salary greater than a given amount AND joiningYear after 2020 (using $and), (c) sort employees by salary descending.

a.

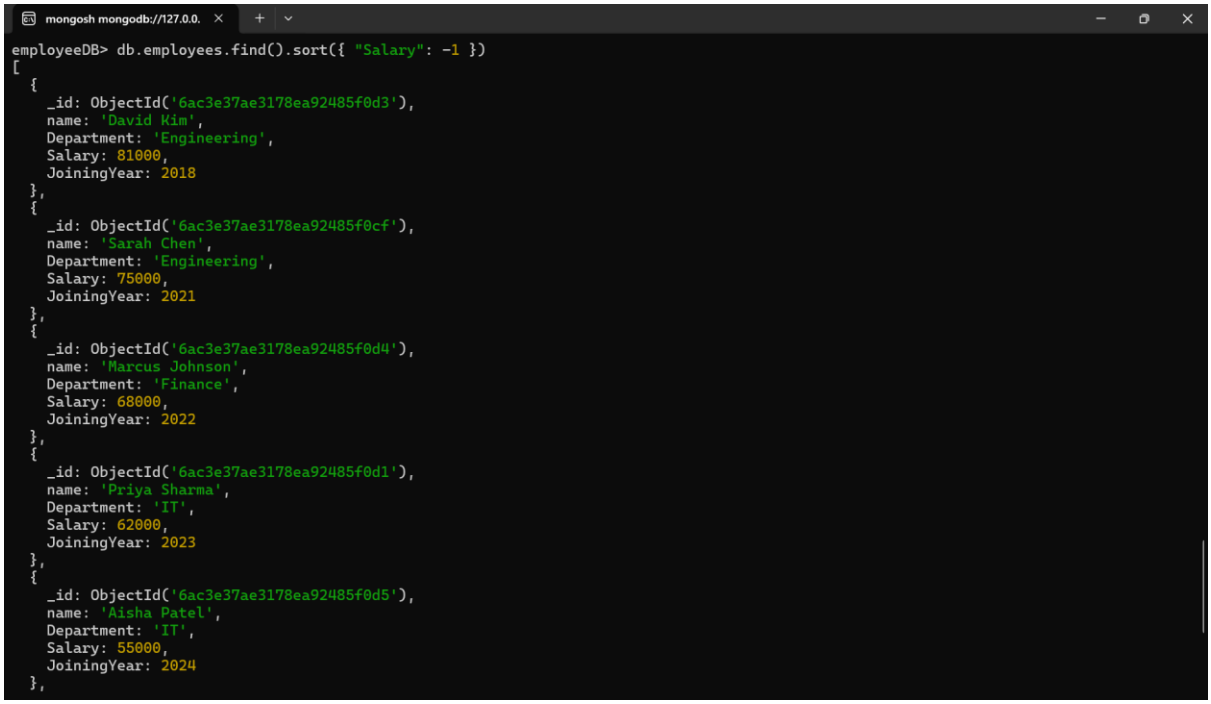
```
employeeDB> db.employees.find({"Department":"Finance"})
[
  {
    _id: ObjectId('6ac3e37ae3178ea92485f0d4'),
    name: 'Marcus Johnson',
    Department: 'Finance',
    Salary: 68000,
    JoiningYear: 2022
  }
]
employeeDB>
```

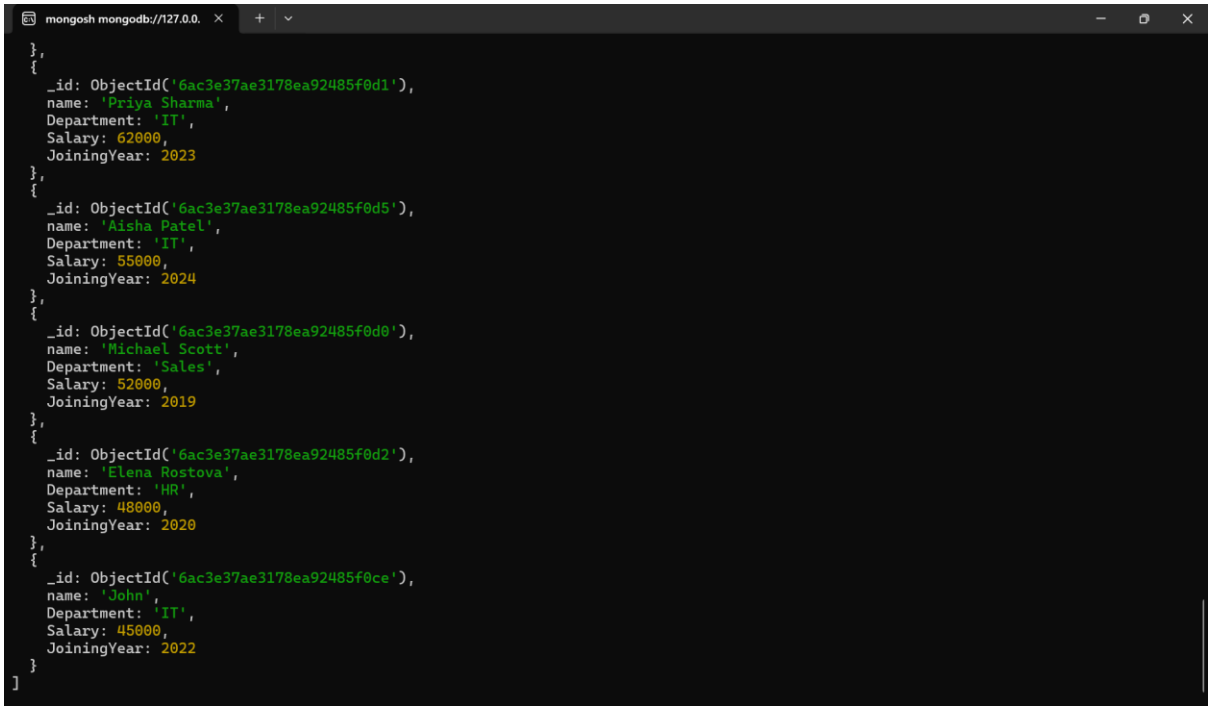


b.

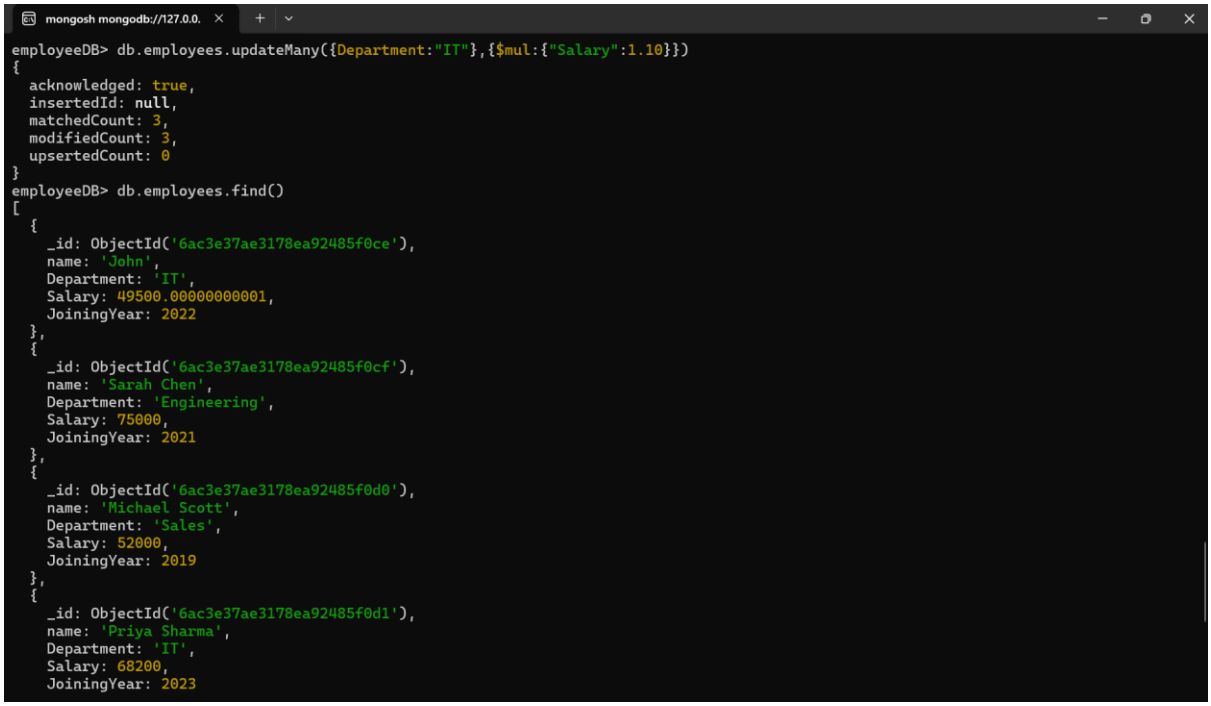

mongosh mongodb://127.0.0.
employeeDB> db.employees.find({
$and: [
{ "Salary": { $gt: 50000 } },
{ "JoiningYear": { $gt: 2020 } }
]
})
[
{
_id: ObjectId('6ac3e37ae3178ea92485f0cf'),
name: 'Sarah Chen',
Department: 'Engineering',
Salary: 75000,
JoiningYear: 2021
},
{
_id: ObjectId('6ac3e37ae3178ea92485f0d1'),
name: 'Priya Sharma',
Department: 'IT',
Salary: 62000,
JoiningYear: 2023
},
{
_id: ObjectId('6ac3e37ae3178ea92485f0d4'),
name: 'Marcus Johnson',
Department: 'Finance',
Salary: 68000,
JoiningYear: 2022
},
{
_id: ObjectId('6ac3e37ae3178ea92485f0d5'),
name: 'Aisha Patel',
Department: 'IT',
Salary: 55000,
JoiningYear: 2024
}
]
employeeDB>

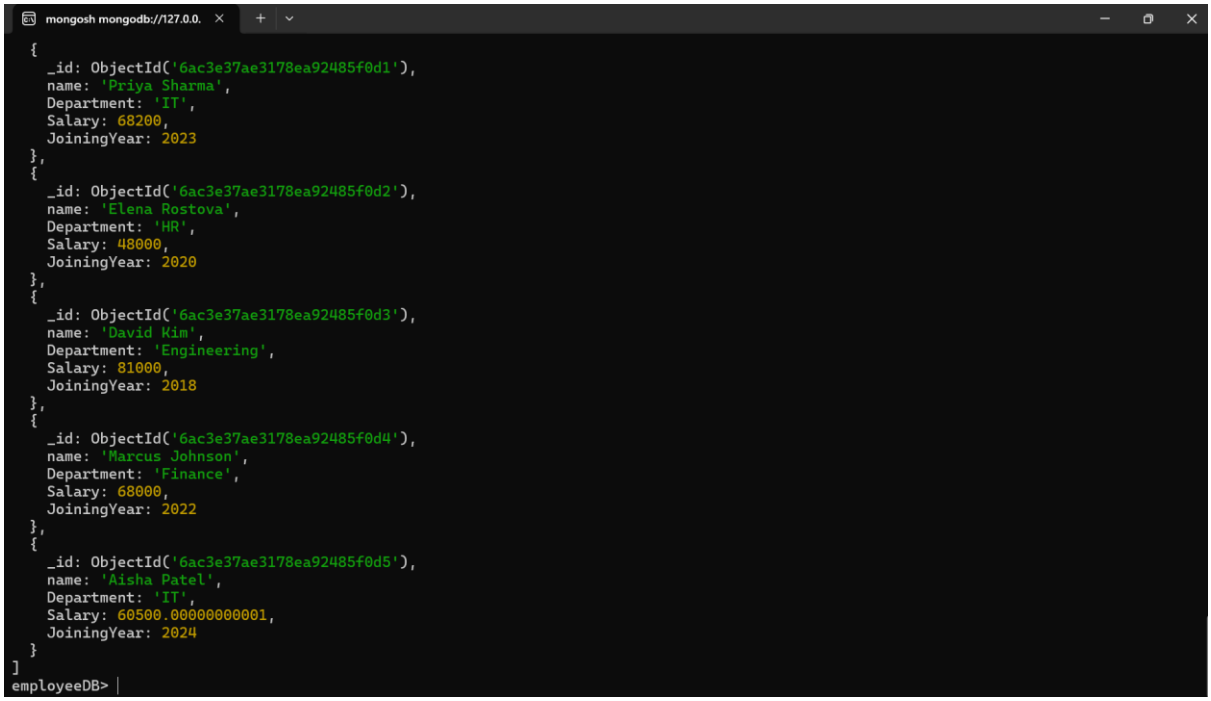
C.

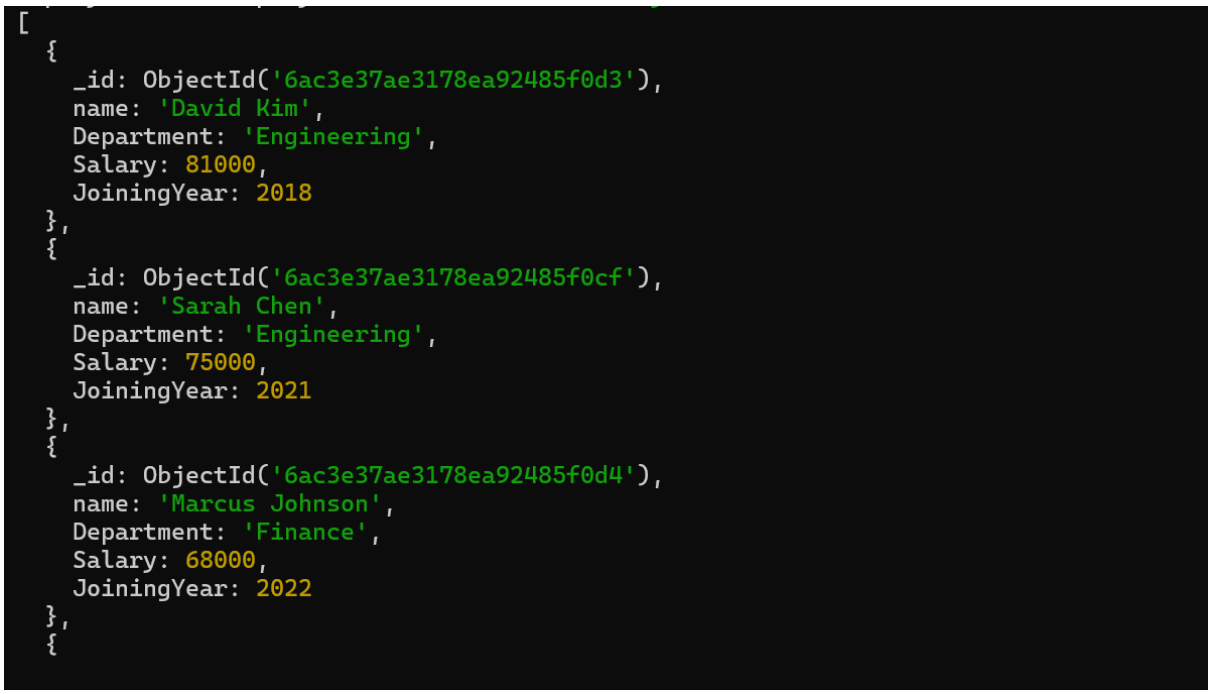

mongosh mongodb://127.0.0.
employeeDB> db.employees.find().sort({ "Salary": -1 })
[
{
_id: ObjectId('6ac3e37ae3178ea92485f0d3'),
name: 'David Kim',
Department: 'Engineering',
Salary: 81000,
JoiningYear: 2018
},
{
_id: ObjectId('6ac3e37ae3178ea92485f0cf'),
name: 'Sarah Chen',
Department: 'Engineering',
Salary: 75000,
JoiningYear: 2021
},
{
_id: ObjectId('6ac3e37ae3178ea92485f0d4'),
name: 'Marcus Johnson',
Department: 'Finance',
Salary: 68000,
JoiningYear: 2022
},
{
_id: ObjectId('6ac3e37ae3178ea92485f0d1'),
name: 'Priya Sharma',
Department: 'IT',
Salary: 62000,
JoiningYear: 2023
},
{
_id: ObjectId('6ac3e37ae3178ea92485f0d5'),
name: 'Aisha Patel',
Department: 'IT',
Salary: 55000,
JoiningYear: 2024
},



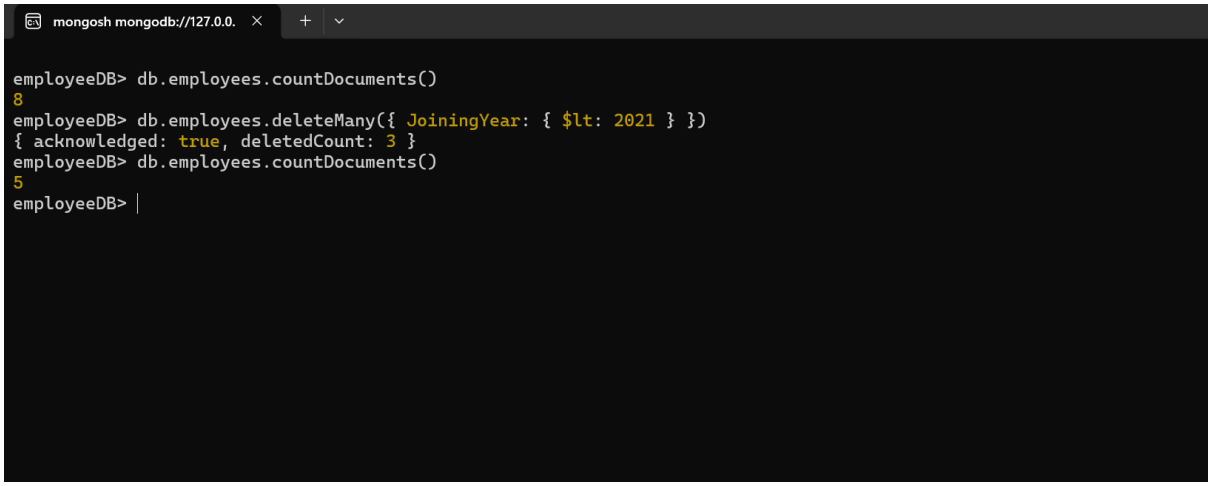
3. Perform an update that gives a 10% raise ($mul or $inc) to all employees in one department, and verify the change with a find() query before and after.







4. Delete all employees who joined before a given year, then use countDocuments() to confirm the correct number of documents remain, and submit all commands with their output screenshots.


mongosh mongodb://127.0.0.
employeeDB> db.employees.countDocuments()
8
employeeDB> db.employees.deleteMany({ JoiningYear: { $lt: 2021 } })
{ acknowledged: true, deletedCount: 3 }
employeeDB> db.employees.countDocuments()
5
employeeDB>